

APIs com Next.js

- O que são APIs
- Route Handlers no Next.js
- Comunicação client ↔ API
- Exemplo prático: Login com POST /login

O que é uma API?

API (Application Programming Interface) é um conjunto de regras que permite que dois sistemas se comuniquem. É a "ponte" entre o frontend e o backend.

01

Requisição

Cliente envia dados via HTTP (GET, POST, PUT, DELETE...)

02

Processamento

Servidor recebe, processa e consulta banco ou lógica

03

Resposta

Servidor retorna JSON com dados ou mensagem de erro

Métodos HTTP

GET	Buscar / listar dados	GET /api/users
POST	Criar novo recurso ou executar ação	POST /api/login
PUT	Atualizar recurso completo	PUT /api/users/1
PATCH	Atualizar parcialmente	PATCH /api/users/1
DELETE	Remover recurso	DELETE /api/users/1

Route Handlers no Next.js

App Router (Next.js 13+)

Estrutura de Pastas

```
app/  
  api/  
    login/  
      route.ts  
    users/  
      route.ts
```

```
// app/api/hello/route.ts
```

```
import { NextRequest, NextResponse } from  
  'next/server'  
  
export async function GET(req: NextRequest) {  
  return NextResponse.json({  
    message: 'Olá, Next.js!'  
  })  
}
```



O arquivo route.ts dentro de qualquer pasta em /app/api define um endpoint REST automaticamente.

Comunicação Client → API

Como chamar endpoints a partir do frontend Next.js

```
// Exemplo
```

```
async function fetchUsers() {  
  const res = await fetch('/api/users', {  
    method: 'GET'  
  })  
  
  if (!res.ok) throw new Error('Erro ao buscar usuários')  
  
  const data = await res.json()  
  return data  
}
```

API de Login

app/api/login/route.ts – POST /api/login

```
import { NextRequest, NextResponse } from 'next/server'

// ⚠ Em produção, busque do banco

const USERS = [{ email: 'admin@email.com', password: '1234' }]

export async function POST(req: NextRequest) {

  const { email, password } = await req.json()

  const user = USERS.find(u => u.email === email && u.password === password)

  if (!user) return NextResponse.json({ error: 'Credenciais inválidas' }, { status: 401 })

  return NextResponse.json({ success: true, message: 'Login realizado!' })

}
```

Chamando a API de Login

Frontend faz POST /api/login com email e senha

```
// components/LoginForm.tsx

async function handleLogin(email: string, password: string) {

  const res = await fetch('/api/login', {

    method: 'POST',

    headers: { 'Content-Type': 'application/json' },

    body: JSON.stringify({ email, password })

  })

  const data = await res.json()

  if (!res.ok) { alert(data.error); return }

  alert('✅ ' + data.message)

}
```

Fluxo Completo de Autenticação

